

Single-Node 2×A100 Parallelism Experiment Report

DP vs TP vs PP on NVLink — Megatron-LM mock-data training study

Lab path: /root/working/dp-tp-pp-lab | Generated from measured logs (not synthetic estimates)

1. Executive Summary

This report compares **Data Parallel (DP=2)**, **Tensor Parallel (TP=2)**, and **Pipeline Parallel (PP=2)** against a single-GPU baseline on one Vast.ai node with **2× NVIDIA A100-SXM4-40GB** connected by **NV12** (12 NVLinks). All runs use the same GPT-like Megatron model, BF16, mock data, global batch 64 (except DP weak scaling), sequence length 2048, 20 warmup + 100 measured iterations.

Key findings. DP2 delivers **1.96×** aggregate throughput (98% strong-scaling efficiency) with almost no communication overhead on NVLink. TP2 is only **1.21×** faster than 1GPU because NCCL collectives consume ~37% of profiled CUDA kernel time, but it cuts peak VRAM by ~48%. PP2 reaches up to **2.08×** at microbatch=4 as pipeline bubble shrinks relative to useful work and larger microbatches improve GEMM efficiency; at microbatch=1 it already saves ~44% VRAM versus the full replica.

2. Experimental Setup

2.1 Hardware & software

Item	Value
GPUs	2 × NVIDIA A100-SXM4-40GB (40 GB each)
Topology	GPU0 ↔ GPU1 = NV12 (12× 25 GB/s NVLink)
Driver / CUDA	580.159.03 / toolkit 13.1
PyTorch	2.10.0a0+a36e1d39eb.nv26.01 (NGC)
Megatron-LM	commit be85fc5 (Megatron Core 0.20)
Precision / data	BF16 / --mock-data (NullTokenizer)
Entry point	pretrain_gpt.py via torch.distributed.run

2.2 Frozen model configuration

```
num_layers=24 hidden_size=2048 ffn_hidden_size=8192 num_attention_heads=16
seq_length=2048 vocab_size≈32000 micro_batch_size=1 (default) global_batch_size=64
train_iters=120 (warmup≈20, measured≈100) optimizer=Adam LR cosine 1.5e-4→1.5e-5
```

Tokens per optimizer step = global_batch × seq_len = **64 × 2048 = 131,072** (262,144 for DP weak scaling with global_batch=128). Throughput = tokens_per_step / mean_step_time (warmup excluded). The 1GPU configuration peaked at **~33.9 GB** allocated and fit without shrinking the model.

2.3 What each parallelism mode does

DP=2: each GPU holds a full model replica and trains on different microbatches; gradients are synchronized with AllReduce once per step.

TP=2: one model replica is sharded across GPUs inside each Transformer layer; AllReduce/collectives run many times per layer during forward and backward.

PP=2: layers are split into two stages (≈12 layers each); activations and gradients move between stages via P2P SendRecv; idle time appears as a pipeline bubble.

3. NVLink Communication Baseline (NCCL)

Before training, raw GPU↔GPU bandwidth was measured with NVIDIA nccl-tests (AllReduce / AllGather / ReduceScatter, 8 MiB → 1 GiB, 2 GPUs). This establishes the physical capability of the NV12 link that DP/TP/PP will use.

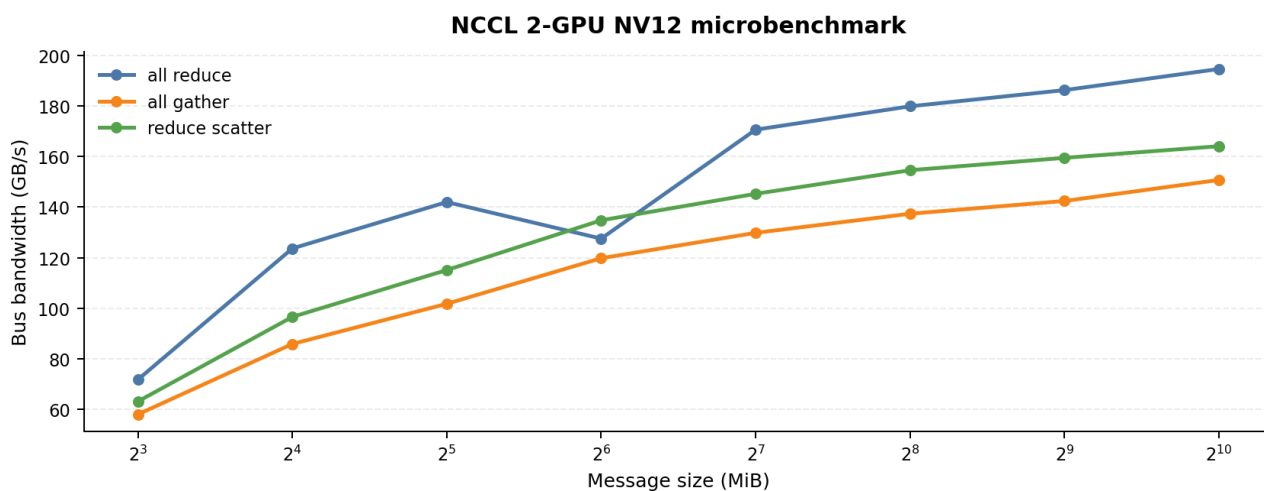


Figure 1. NCCL bus bandwidth vs message size on NV12.

Collective (1 GiB)	time (μs)	algbw (GB/s)	busbw (GB/s)	Relevance
AllReduce	5518	194.6	194.6	DP grad sync
AllGather	3562	301.5	150.7	TP-style ops
ReduceScatter	3272	328.2	164.1	TP-style ops

Observation: large-message AllReduce busbw ≈ **195 GB/s** on this pair — high enough that infrequent DP synchronization is cheap, but not free enough to hide TP's many small/medium collectives inside every layer.

4. Main Training Results

All primary configs use identical architecture, sequence length, precision, and global batch 64. Metrics below are computed from Megatron iteration logs after discarding the first 20 warmup steps. Peak memory uses Megatron's reported max allocated.

Config	GPUs	TP	PP	DP	Tok/s	Speedup	Tok/s/GPU	Step ms	Peak GB	Util%
1GPU	1	1	1	1	16172	1.00×	16172	8105	33.9	96
DP2	2	1	1	2	31666	1.96×	15833	4139	33.9	94/94
TP2	2	2	1	1	19620	1.21×	9810	6680	17.5	74/95
PP2 mbs1	2	1	2	1	28453	1.76×	14226	4607	18.8	94/88
PP2 mbs2	2	1	2	1	32376	2.00×	16188	4048	23.2	93/92
PP2 mbs4	2	1	2	1	33590	2.08×	16795	3902	31.6	93/93

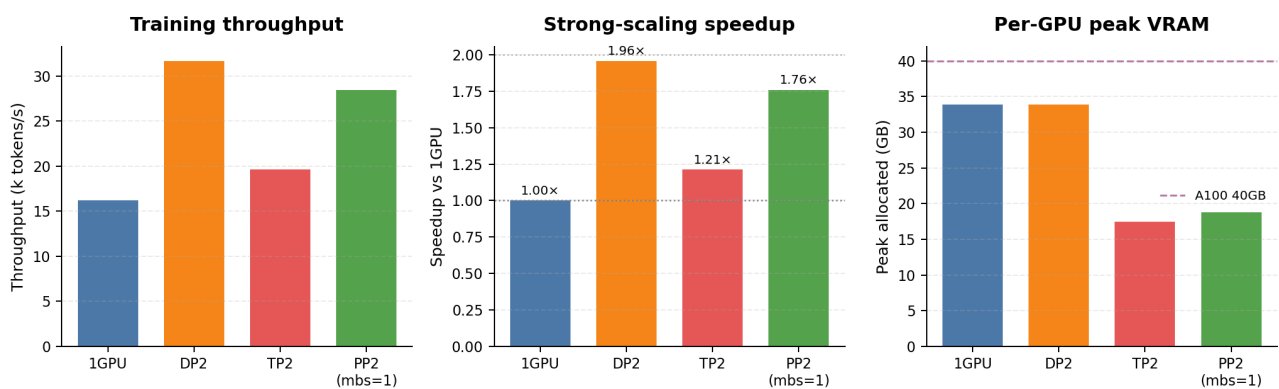


Figure 2. Throughput, speedup, and peak VRAM for 1GPU / DP2 / TP2 / PP2(mbs=1).

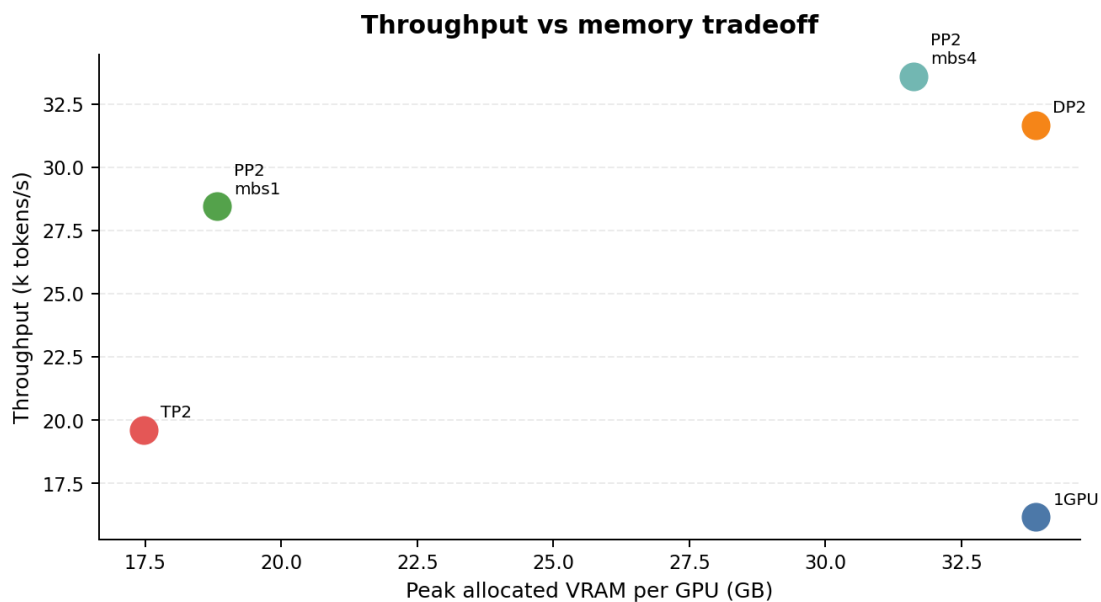


Figure 3. Throughput–memory tradeoff. DP maximizes tokens/s; TP minimizes VRAM; PP sits in between and can approach DP throughput with enough microbatches.

5. Data Parallel (DP=2)

5.1 Strong scaling (same global batch)

With `global_batch` fixed at 64, DP2 reduces mean step time from **8105 ms** to **4139 ms** and raises throughput from **16172** to **31666** tokens/s (**1.96×**, efficiency **97.9%**). Each GPU still stores a full replica (~33.9 GB peak), so DP does **not** solve capacity limits; it replicates work across data shards.

5.2 Weak scaling (2× global batch)

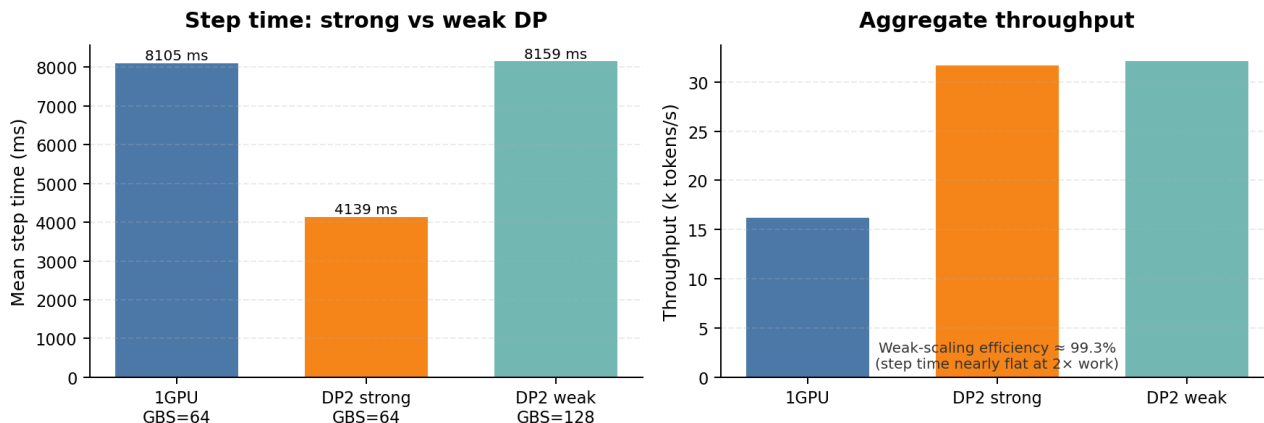


Figure 4. DP strong vs weak scaling. Ideal weak scaling keeps step time flat when GPUs and work both ×2.

Raising `global_batch` to 128 on 2 GPUs yields step time **8159 ms** vs **8105 ms** on 1GPU — weak-scaling efficiency **99.3%**. Aggregate tokens/s rises to **32128**. This is the regime that usually extends cleanly to multi-node DP when interconnect is adequate.

6. Tensor Parallel (TP=2)

TP2 keeps one logical replica but shards tensors across GPUs. Throughput is **19620** tokens/s (**1.21×** vs 1GPU) — faster than one GPU, but far behind DP2. Peak allocated memory falls to **17.5 GB** (~48% savings). GPU0 average SM util is lower (74%) than GPU1 (95%), consistent with communication/wait imbalance under frequent collectives.

Why TP underperforms DP here: the model already fits on one A100, so TP's benefit is mostly memory headroom, not necessary for capacity. Every linear/attention partition pays NVLink collectives; Nsight shows AllReduce alone at ~37% of CUDA kernel time. DP pays AllReduce only once per step (~1.3%).

7. Pipeline Parallel (PP=2)

PP2 splits the 24 layers into two stages. With `microbatch=1` (64 microbatches/step), throughput is **28453** tokens/s (**1.76×**) and peak memory **18.8 GB** (~44% vs 1GPU). Unlike TP, PP does not make a single layer faster; it overlaps different microbatches on different stages.

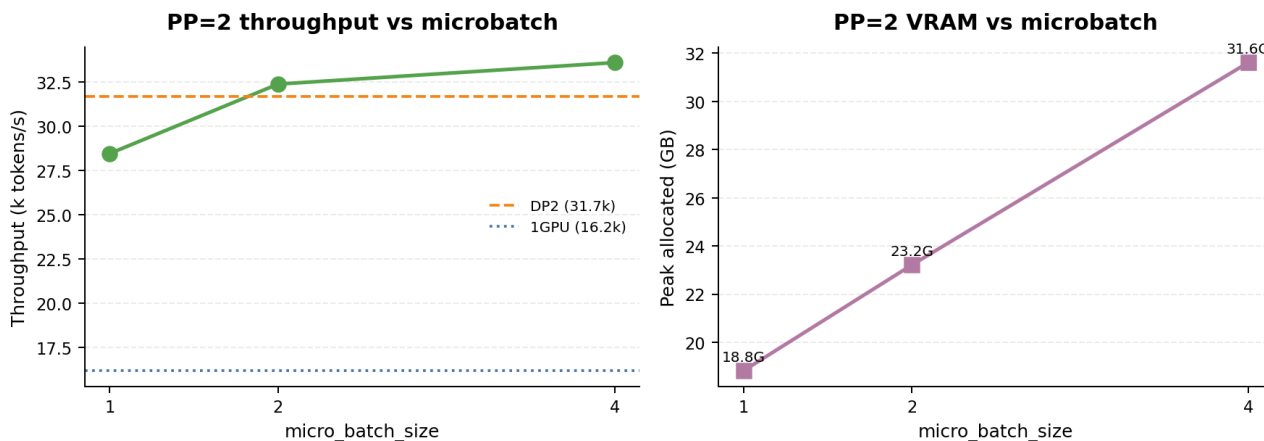


Figure 5. PP=2 microbatch sweep: throughput rises with larger microbatches while activation memory grows.

micro_batch	#microbatches	Tok/s	Speedup	Step ms	Peak GB
1	64	28453	1.76×	4607	18.8
2	32	32376	2.00×	4048	23.2
4	16	33590	2.08×	3902	31.6

Tradeoff: more microbatches reduce bubble fraction $\approx (PP-1)/N_{\text{micro}}$, while larger microbatches improve GEMM efficiency and raise activation memory. On this 2-stage pipeline, increasing microbatch from 1→4 raised tokens/s by **18%** even though bubble fraction grew ($1/64 \rightarrow 1/16$), because compute intensity improved enough to dominate.

8. Nsight Systems Communication Evidence

Short profiles were captured for DP2, TP2, and PP2 (see profiles/nsys_*.nsys-rep). Kernel summaries quantify how much GPU time is spent in NCCL versus math kernels.

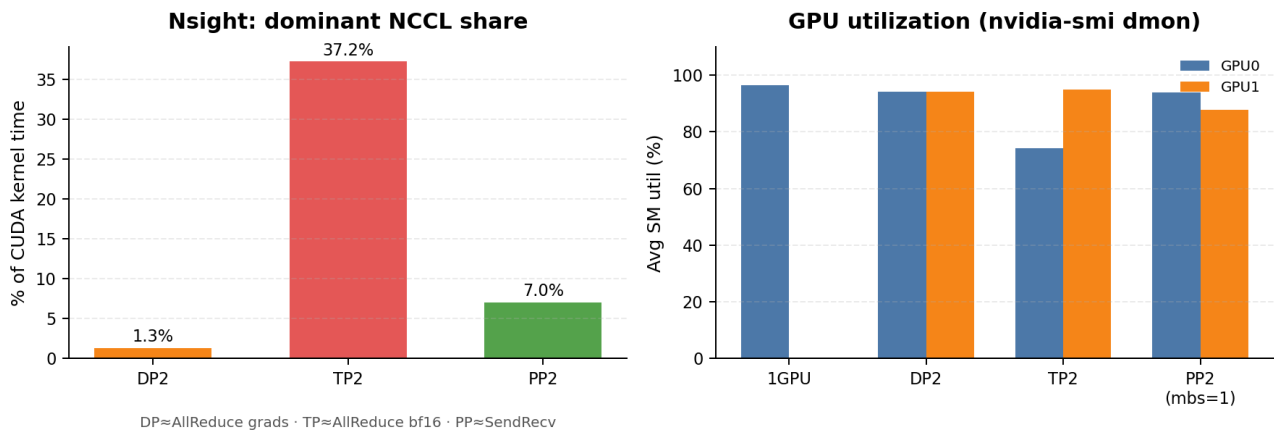


Figure 6. Left: share of profiled CUDA kernel time in the dominant NCCL kernel. Right: average SM utilization during full benchmarks.

Config	Dominant NCCL kernel	% kernel time	Timeline intuition
DP2	AllReduce (f32 grads)	1.3%	Long compute → short sync
TP2	AllReduce (bf16)	37.2%	GEMM ↔ NCCL interleaved
PP2	SendRecv	7.0%	Staggered stages + bubble

Takeaway: communication frequency — not just peak NVLink bandwidth — decides efficiency. DP communicates rarely and scales well. TP communicates constantly and leaves throughput on the table even on NV12. PP communicates at stage boundaries and exposes idle bubbles that microbatching amortizes.

9. Answers to the Study Questions

1. How much faster is DP=2 than one GPU?

1.96× aggregate tokens/s (31666 vs 16172).

2. What is DP scaling efficiency?

Strong: **97.9%** (= speedup/2). Weak (GBS 64→128): **99.3%**.

3. For the same global batch, is TP=2 faster than one GPU?

Yes, modestly — **1.21×** (19620 tokens/s).

4. Is TP=2 more or less efficient than DP=2?

Less efficient (1.21× vs 1.96×; 9810 vs 15833 tok/s/GPU).

5. How much VRAM does TP save?

Peak allocated **33.9 → 17.5 GB** (~48% reduction).

6. How much VRAM does PP save?

At mbs=1: **33.9 → 18.8 GB** (~44%). Savings shrink as microbatch grows (mbs=4 → 31.6 GB).

7. How does PP throughput change with microbatches?

mbs 1→2→4: **28453 → 32376 → 33590** tokens/s.

8. What do DP/TP/PP look like in Nsight?

DP: compute-heavy with rare AllReduce. TP: dense AllReduce between GEMMs (~37% kernel time). PP: SendRecv between offset stage timelines with bubble/idle regions.

9. Which strategy communicates most frequently?

TP (tens of thousands of AllReduce instances in the profile window).

10. Why prefer DP when the model fits on one GPU?

Highest tokens/s with minimal communication; nearly ideal scaling on NVLink in this study.

11. Why might TP still be useful if the model fits?

Halves per-GPU parameter memory, enables larger batches/models later, and composes with DP/PP — at a clear throughput cost from collectives.

12. Why is PP mainly useful for larger models / more GPUs?

It stages layers so models that exceed single-GPU memory can train; deeper pipelines need many microbatches (or virtual stages) to keep bubbles small.

10. Conclusions & Practical Guidance

On this **2×A100 NV12** node with a model that already fits in 40 GB:

- Choose **DP** for maximum training throughput and simplest scaling.
- Choose **TP** when you need per-GPU memory headroom or are building multi-dimensional parallel plans; expect heavy collective traffic even on NVLink.
- Choose **PP** to split depth for capacity; tune microbatch size — too small leaves bubble/GEMM inefficiency, too large blows activation memory.
- Always measure NCCL first: good NVLink bandwidth is necessary but not sufficient if your parallel strategy communicates every layer.

11. Reproducibility

```
cd /root/working/dp-tp-pp-lab
bash run_all.sh
# artifacts: results/*.csv logs/*.log profiles/nsys_*.nsys-rep analysis/figures/
python3 analysis/generate_report.py # regenerates this PDF
```

End of report — all numeric claims are taken from `results/training_results.csv`, `results/nccl_results.csv`, Megatron logs, and Nsight kernel summaries.